

Dynamic Multi-Byte Prediction With Hierarchical Language Models

Abraham Toluwase Owodunni¹ Chibuzor Okocha² Christan Grant² Tomasz Limisiewicz³
Sachin Kumar¹

¹The Ohio State University ²University of Florida ³ University of Washington
owodunni.1@osu.edu

Abstract

Byte-level hierarchical language models (LMs) have recently emerged as a robust alternative to their popular counterparts that use subword tokenization. However, generating one byte at a time remains a bottleneck for inference speed. To address this, we introduce multi-byte prediction (MBP) for hierarchical models, which generates multiple bytes in parallel, speeding up inference with minimal performance impact and no additional parameters. MBP builds on the popular multi-token prediction (MTP) paradigm with two crucial innovations. First, we introduce a variable-length prediction window that aligns with the latent tokens, or segments, of a hierarchical LM. Second, we implement a novel attention-masking scheme that enables parallel byte prediction without violating causality. We show that multi-byte prediction strikes a Pareto-optimal trade-off across multiple generative tasks, instruction following, question answering, summarization, and machine translation, achieving the best trade-off between performance and inference throughput. Code for our experiments will be released at github.com/skai-research/lca-multibyte.

1 Introduction

Subword tokenization has been a central design choice in modern language models, enabling efficient text compression and open-vocabulary generation (Sennrich et al., 2016; Kudo, 2018). However, subword tokenizers, due to their predetermined fixed vocabulary, may overfragment rare words, encode language- and script-specific biases, and generalize poorly across domains, non-English settings, and evolving scenarios (Mielke et al., 2021; Menschikov et al., 2025). Byte-level language modeling (Xue et al., 2022; Wang et al., 2024; Yu et al., 2023) offers an appealing alternative: by operating directly on raw bytes, models can eliminate tokenizer-specific assumptions and represent any text sequence.

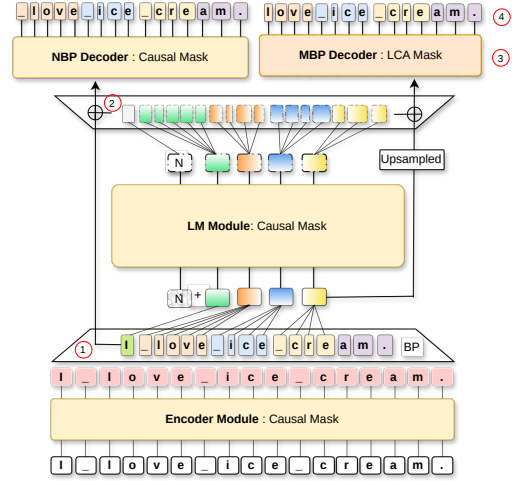


Figure 1: **Overview of the MBP architecture.** An input byte sequence is first processed by the model, and the MBP head uses the LCA mask (Figure 2) to predict multiple future bytes in parallel from a single head. Colored bands denote learned segment boundaries. See Appendix A for a detailed description of several parts in this figure.

Despite these advantages, byte-level modeling remains computationally expensive (Tay et al., 2022; Yu et al., 2023). Byte sequences are substantially longer than subword sequences, which increases the cost of Transformer attention and makes autoregressive decoding slower (Xue et al., 2022; Wang et al., 2024; Slagle, 2024). Recent hierarchical byte-level models attempt to address this issue by compressing byte streams into shorter latent sequences (Slagle, 2024; Pagnoni et al., 2025; Hwang et al., 2025; Owodunni et al., 2025) by incorporating heuristic or learned tokenization mechanisms that reduce the effective sequence length before applying deeper Transformer computation. These approaches show that byte-level models benefit from operating over higher-level units rather than individual bytes alone. However, most hierarchical designs use latent tokenization primarily for representation efficiency: the model compresses

the input, processes the shorter sequence, and then decodes back to bytes (Hwang et al., 2025; Neite-meier et al., 2025). Inference still happens one byte at a time.

In parallel, multi-token prediction (MTP) has emerged as a promising approach for accelerating language model decoding. Early work introduced the idea of predicting multiple future tokens in a single forward pass to reduce decoding latency (Stern et al., 2018; Qi et al., 2020), and more recent methods have shown that this objective also improves sample efficiency at scale (Gloeckle et al., 2024). Existing MTP approaches typically rely on multiple prediction heads attached to the model trunk (Cai et al., 2024; Ankner et al., 2024) or on smaller draft models that propose candidates for the main model to verify (Leviathan et al., 2023; Chen et al., 2023; Li et al., 2024). While these designs can improve inference speed, they have two important limitations. First, predicting more future tokens typically requires additional heads or auxiliary parameters for every token, increasing memory cost. Second, fixed-offset prediction treats the next n tokens as a static horizon, ignoring the fact that language has variable local structure: some regions are easily predictable as coherent chunks, while others require finer-grained sequential modeling.

This paper proposes Latent Causal Attention (LCA), a dynamic multi-byte prediction method for hierarchical language models. Our key idea involves the use of learned byte segments not only for compression, but also as the unit of multi-byte generation. Instead of assigning one prediction head to each future byte, LCA uses a single multi-byte decoder equipped with a boundary-aware causal mask that allows each byte in a segment to attend to previous segments while preventing dependence on other bytes within the same segment. This preserves global autoregressive causality while enabling all bytes in a predicted segment to be generated in parallel. This design replaces the fixed-offset prediction of prior MTP methods with variable-length, segment-aligned generation. LCA removes the need for additional prediction heads as the number of generated tokens increases and provides a natural extension of hierarchical byte-level modeling to inference acceleration. We describe our methodology in detail in Section 3. We then present our findings and extensive analyses in Sections 5 and 6, respectively.

2 Background

2.1 Multi-token Prediction

Multi-token prediction (MTP) has emerged as a way to improve both sample efficiency and inference speed of autoregressive language models by predicting multiple future subword tokens in parallel, as opposed to one token at a time (Gloeckle et al., 2024; Cai et al., 2024). Given a sequence $x_1, \dots, x_T$, MTP is trained by minimizing $\mathcal{L}_{\text{MTP}} = -\sum_t \log P_\theta(x_{t+1:t+n} | x_{1:t})$ where θ parameterizes the LM and n is the number of future tokens to be predicted. To make this objective tractable, n dedicated prediction heads (typically multiple MLP layers or independent transformer blocks followed by a shared unembedding matrix) are used to predict each of the n future tokens independently and in parallel. Each head operates on a shared hidden representation z_t encoding $x_{1:t}$. Under this conditional independence assumption, the joint probability factorizes and loss becomes:

$$\mathcal{L}_{\text{MTP}} = -\sum_t \sum_{i=1}^n \log P_\theta(x_{t+i} | z_t) \quad (1)$$

Since different parts of a sequence exhibit varying levels of surprisal, always predicting a fixed n tokens per forward pass makes this approach rigid and unable to adapt to the local difficulty of the text. Moreover, this design also introduces a parameter bottleneck: each additional future token requires its own prediction head, linearly increasing the prediction head’s parameter count with n .

2.2 Hierarchical Language Models

Hierarchical byte-level language models have recently emerged as a viable alternative to subword-based language models. In this work, we build on the architectural design of FlexiTokens (Owodunni et al., 2025), introducing multiple modifications to support dynamic multi-byte prediction. Our hierarchical model comprises four components (Figure 1): an encoder module, a language modeling (LM) module, a boundary predictor, and a decoder module. We note that our proposed method is generalizable to other hierarchical LM (Pagnoni et al., 2025; Hwang et al., 2025) architectures since they follow a similar design.

Encoder Module. This module maps an input byte sequence $x_1, \dots, x_T$ to hidden states $h_e \in \mathbb{R}^{T \times d}$ via transformer layers with a causal attention

mask. h_e serves both as a rich contextual representation and as a residual connection for downstream modules. We then feed h_e into a boundary predictor $\mathcal{B}$, which produces boundary probabilities $\hat{B} \in [0, 1]^T$, indicating positions where a segment should end. A discrete label is sampled from these probabilities using Gumbel Sigmoid trick to maintain differentiability (?). At positions where the label is 1, h_e is downsampled via mean pooling with all preceding positions where the label is 0 to form a shortened hidden representation $h_\downarrow$ or *latent tokens*, with sequence length strictly less than that of h_e . The boundary predictor in this module is jointly optimized during training; we use a boundary predictor loss $\mathcal{L}_{BP}$ that controls the compression of the input sequence. Following Owodunni et al. (2025), we define:

$$\mathcal{L}_{BP} = \max\left(\frac{k}{T} - \alpha, 0\right) + \max\left(\beta - \frac{k}{T}, 0\right)$$

where k is the number of boundaries predicted as 1. Hyperparameters α and β define upper and lower bounds on k/T , respectively, such that the compression rate is considered optimal when $\beta \leq \frac{k}{T} \leq \alpha$.

Language Modeling Module. A learned [BOS] token embedding is appended to $h_\downarrow$ before being passed through the LM module’s transformer stack to output hidden vectors $h'_\downarrow$. The [BOS] token shifts the shortened sequence forward by one, ensuring that the representation at each latent token position conditions only on preceding tokens, thereby preserving causality. $h'_\downarrow$ is subsequently upsampled via duplication to match the original byte sequence length and passed to the decoder.

Decoder Module. This module produces byte-level predictions. We add a residual connection $r_1 = h_e$ from the encoder to the upsampled output of the LM module. The result is passed through a small stack of transformer layers, followed by an unembedding layer forming the primary next-byte prediction head, optimized with cross-entropy loss.

3 Multi-Byte Prediction with Latent Causal Attention

Here we describe our modifications to the hierarchical LM architecture to enable multi-byte prediction. We decouple the decoder module into only two heads: a standard next-byte prediction head (same as before) and a single multi-byte prediction head

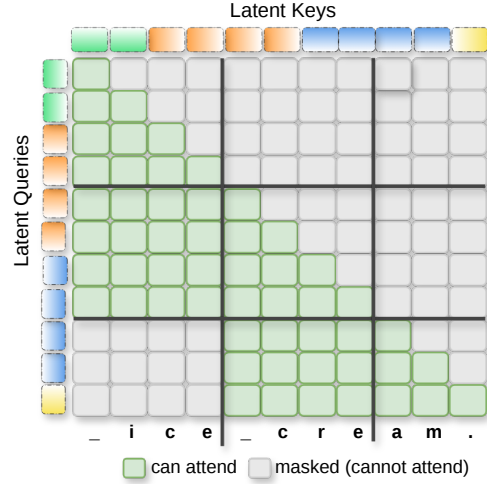


Figure 2: **The Latent Causal Attention (LCA) mask.** Each row corresponds to a query byte and each column to a key byte. A query at position i in segment s_i may attend to itself, to earlier positions within its own segment, and to all bytes in the immediately preceding segment s_{i-1} . Attention to later positions in the same segment and to segments further in the past is masked out. Colored bands denote segment boundaries and match those in Figure 1. This mask preserves autoregressive causality at the segment level while enabling all bytes in a group to be predicted in parallel from a single decoder head.

(MBP; Figure 1 ③). The MBP head is trained to predict all bytes in a latent token in parallel, except the first which is predicted by the next-byte head. Since the number of bytes in each latent token is variable, a fixed number of independent prediction heads, typically used in the MTP literature, is impractical. Instead, the MBP head consists of transformer layers.

The input to the MBP head is the duplicated upsampled output of the LM module. We add a residual connection $r_2 = h_\downarrow$ from the downsampled representation to each of these vectors.

Since the LM module shifts the latent tokens by one position, this input only carries information about the bytes from the previous latent token. For each transformer layer in the MBP head, we construct an attention mask (Figure 2) such that each position may attend to itself, to earlier positions within the latent token, and to all bytes in the immediately preceding latent token. Intra-token attention does not violate causality at inference time, as the input to each position within the same token are duplicates plus residuals of the previous latent

Algorithm 1 Multi-byte Prediction with LCA

Require: Prompt $x_{1:t}$, boundary predictor $\mathcal{B}$, threshold τ , #MBP candidates n

```
1: while not done do
2:    $h_e \leftarrow \text{EncoderModule}(x_{1:t});$  cache residual  $r_1 \leftarrow h_e$ 
3:    $b \leftarrow \mathcal{B}(h_e)$  ▷ Predict boundary
4:   if  $b = 1$  or no cached  $h_\downarrow$  then
5:      $h_\downarrow \leftarrow \text{Downsample}(h_e);$  cache residual  $r_2 \leftarrow h_\downarrow$ 
6:      $h'_\downarrow \leftarrow \text{LMModule}(h_\downarrow)$ 
7:   end if
8:    $h_1 \leftarrow \text{NTLayers}(\text{Upsample}(h'_\downarrow) + r_1)$ 
9:    $h_2 \leftarrow \text{Upsample}(h'_\downarrow + r_2)$  ▷ Residual added before upsample
10:   $h_2 \leftarrow [h_2; \underbrace{h_2[-1], \dots, h_2[-1]}_{n-2 \text{ copies}}]$  ▷ Duplicate last token  $n-2$  times
11:   $s \leftarrow$  start index of previous segment boundary
12:   $h_2 \leftarrow \text{MBPLayers}(h_2[s:], \text{segment mask})$  ▷ Only prev. + current segment
13:   $t_1 \sim \text{Head}(h_1[-1])$  ▷ NT: always accepted
14:   $t_{2:n+1} \sim \text{Head}(h_2[-(n-1):])$  ▷ MBP:  $n$  speculative
15:  Accept  $t_{2:k}$  while  $P(t_k) \geq \tau$ ; discard rest
16:  Append accepted bytes to  $x_{1:t}$ 
17: end while
```

token. We call this mask *Latent Causal Attention* (LCA) and our overall method as LCA-MBP. The output of the transformer layers is multiplied by an unembedding matrix to predict byte logits over which the cross-entropy loss is computed to predict target bytes shifted by two positions (Figure 1 ④). Both decoder heads share a single unembedding matrix.

The overall training objective combines the next-byte prediction loss with the multi-byte prediction loss:

$$\mathcal{L}_{\text{LM}} = -\lambda_0 \sum_t \log P_\theta(x_{t+1} | x_{1:t}) - \lambda_1 \sum_t \log P_\theta(x_{t+2} | x_{i:t}) \quad (2)$$

where i is the starting index of the x_t 's immediately previous latent token. λ_0 and λ_1 are hyperparameters. Unlike standard MTP, which requires separate prediction heads, LCA achieves this independence through the latent attention mask and byte segments pooling, while using a single set of shared transformer layers. The final loss we train this model with is $\mathcal{L}_{\text{LM}} + \lambda_2 \mathcal{L}_{\text{BP}}$, where λ_2 is another hyperparameter.

3.1 Inference Algorithm

We present our inference methodology in Algorithm 1. At inference time, the prompt $x_{1:t}$ is first

passed through the encoder module to produce a hidden representation h_e , from which we store a residual connection $r_1 = h_e$. The boundary predictor is then applied to h_e to identify boundaries.

To generate the first token, if the boundary predicted at position x_t is 1, or no cached LM module state exists, we downsample h_e to obtain $h_\downarrow$ (Line 5), store a residual $r_2 = h_\downarrow$, and pass $h_\downarrow$ through the LM module to produce $h'_\downarrow$ (Line 6). Otherwise, the cached $h'_\downarrow$ from the previous step is reused, avoiding a full LM forward pass.

To predict the next byte, we add the residual r_1 to $h'_\downarrow$, and process the result through the next-byte head (Line 8). To predict the next n bytes with the MBP head, we add the residual r_2 to $h'_\downarrow$ before upsampling to obtain h_2 . We then slice h_2 to retain only the bytes belonging to the current and previous latent tokens, and pass this window through the MBP transformer layers with the LCA mask (Lines 9–12).

Byte Acceptance. The output of the next-byte head is always accepted (Line 14). For the MBP head, a predicted token $\hat{x}_{t+i}$ is accepted only if the model's confidence exceeds a predefined threshold:

$$\hat{x}_{t+i} \text{ is accepted} \iff P_\theta(\hat{x}_{t+i} | \cdot) \geq \tau$$

Tokens are accepted from left to right; the first token falling below τ and all subsequent candidates are discarded. This acceptance strategy is applied

uniformly across all MBP methods evaluated in this work, following [Kirchenbauer et al. \(2026\)](#).

4 Experimental Setup

4.1 Baselines

We compare LCA against five baselines spanning flat and hierarchical byte-level architectures.

- **LlamaByte** ([Grattafiori et al., 2024](#)): a vanilla byte-level model implemented with the Llama 3 architecture.
- **SpaceByte** ([Slagle, 2024](#)): a hierarchical byte-level model that uses an external fixed heuristic for segmentation during training. An external fixed heuristic was also used in BLT ([Pagnoni et al., 2025](#)).
- **FlexiTokens (FxT)** ([Owodunni et al., 2025](#); [Hwang et al., 2025](#)): a hierarchical model with a learned boundary predictor that dynamically segments byte sequences into variable-length segments. This boundary predictor is jointly optimized with the language modeling objective during training.
- **MLP-MBP** ([Gloeckle et al., 2024](#)): a multi-byte prediction baseline built on the FxT architecture, which employs n independent MLP heads to predict n future tokens in parallel, one per head, following the setup of [Gloeckle et al. \(2024\)](#) and Medusa ([Cai et al., 2024](#)).
- **Efficient FlexiTokens (Eff-FxT)**: a variant of FxT that uses a more efficient inference algorithm. Specifically, the LM module is only invoked at predicted boundary positions; at all other positions, the cached LM module output is reused. This corresponds exactly to Lines 4–8 in Algorithm 1. We note that HNet ([Hwang et al., 2025](#)) uses a similar generation strategy in their work as well.

To ensure a fair comparison, all models are parameter-matched by scaling each baseline to the same total parameter count, which consequently increases the floating-point operations (FLOPs) of flat models such as LlamaByte and SpaceByte. See Table 2 in Appendix A for the number of layers used across all methods.

4.2 Model Architecture

For all models, we use a hidden dimension of $d_{\text{model}} = 1024$, an LM inner dimension of $d_{\text{inner}} = 4096$, and a context window of 4096. We also use RMSNorm with $\epsilon = 10^{-6}$, a rotary embedding base of 100000, 8 attention heads, 8 KV heads, dropout of 0.1, and attention scaling of 1.0. Unless otherwise stated, we keep these architectural and optimization settings fixed across runs. The main architectural differences across models, therefore, lie in how layers are allocated across the encoder, LM module, next-token (NT) decoder, and multi-byte (MB) decoder. For the plain LlamaByte baseline, all layers are placed in a single stack, while the hierarchical baselines distribute layers across the encoder, LM module, and decoder components. Among these hierarchical variants, the principal difference is the choice of MB prediction head: either no MB head, multiple MLP heads, or a two-layer LCA head.

4.3 Model Training

We follow Llama 3 ([Grattafiori et al., 2024](#)) for the general implementation of all parts of our model, and we detail our configurations in Appendix A. We pretrain a 373M-parameter hierarchical transformer model on 50B bytes from FineWeb-edu English dataset ([Lozhkov et al., 2024](#)). We train with a total batch size of 1048576 bytes per gradient step and a warmup of 2000 steps. We use a learning rate of $3\text{e-}4$ and the Adam optimizer with a cosine learning rate scheduler. We set Adam β_1 and β_2 to 0.9 and 0.999, respectively, while we set the ϵ to $1\text{e-}8$. We employ gradient clipping at 0.25. All models were trained on 4 NVIDIA B200 GPUs.

We pretrain multiple models with different configurations. We use a compression rate of $3\times$. Following ([Owodunni et al., 2025](#)), we achieve this by setting β and α in the BP loss $\mathcal{L}_{BP}$ to 0.261 and 0.333 respectively. We use 3 heads for MLP-MBP and train all models for 1 epoch and we set $\lambda_0 = 1$, $\lambda_1 = 1$ and $\lambda_2 = 10$.

4.4 Downstream Fine-tuning and Evaluation

We perform supervised finetuning on Tulu-3 SFT mix ([Lambert et al., 2024](#)) for 5 epochs. We use the same hyperparameters as pretraining except for the learning rate, which we set to $2\text{e-}4$, batch size to 64, $\lambda_1 = 2$. To evaluate on translation, we separately finetune our pretrained models on the Spanish and French datasets in Opus-100 ([Zhang et al., 2020](#))

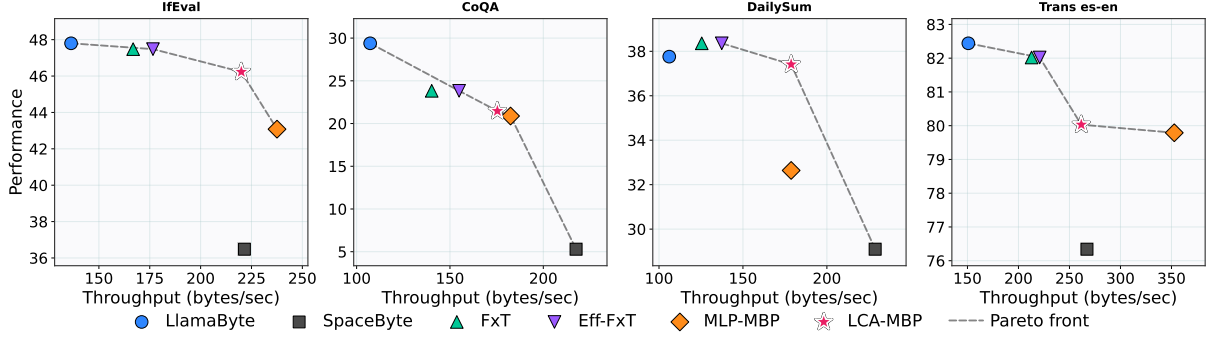


Figure 3: **Performance vs. throughput across four downstream tasks.** Metrics are instruction-loose accuracy on IfEval, F1 on CoQA, ROUGE on CNN/DailyMail, and COMET on Spanish–English translation. LCA (★) lies on the frontier for three of four tasks on the Pareto front, which represents the method with the best tradeoff.

for 2 epochs with the SFT hyperparameters.

We evaluate our models on 4 major generative tasks that effectively measure our speed-to-performance comparison. These include Summarization (CNN/Daily Mail; Hermann et al., 2015), Question Answering (CoQA; Reddy et al., 2019), Machine translation (Opus-100; Zhang et al., 2020), and Instruction Following (IFEval; Zhou et al., 2023). At inference time, we use a temperature of 0.7 and top_p of 0.9 for IFEval, while we use greedy decoding for other tasks since they are factual tasks where the output should closely match a reference. Unless otherwise stated, we set the acceptance threshold $\tau = 0.9$ and candidate bytes $n = 3$ for our main experiments. In all our evaluations, we report wall-clock on a single B200 with a batch size of 1.

5 Results and Analyses

Figure 3 compares all six methods across the four downstream tasks. We highlight two main findings.

LCA-MBP lies on the Pareto front in three of four tasks. Across IFEval, CoQA, and CNN/DailyMail summarization, LCA matches or exceeds the throughput of other methods while retaining task performance close to the strongest non-MBP hierarchical models (FxT and Eff-FxT). On Spanish–English translation, FxT and Eff-FxT achieve slightly higher COMET scores, but at substantially lower throughput; LCA trades a small amount of performance for a meaningful throughput gain. Notably, this result holds even though our pretraining corpus (FineWeb-edu) contains no dedicated translation data, suggesting that the multi-byte prediction structure can transfer to non-English generation after lightweight finetuning. Across all four tasks, LCA achieves this trade-off

with a single decoder head.

Table 1 reports the overall acceptance rates of our LCA and MLP-based multi-byte prediction models. The Pareto comparison in Figure 3 reveals that despite its high acceptance rate, MLP-MBP consistently underperforms LCA on downstream metrics at comparable throughput. This indicates that MLP-MBP is highly confident in its speculative predictions, but those predictions are frequently incorrect.

Model	IfEval	CoQA	DailySum	es-en	fr-en
MLP-MBP	61.91	63.45	58.08	62.67	65.01
LCA-MBP	45.95	46.62	48.89	51.68	49.74

Table 1: Byte acceptance rates of MLP and LCA multi-byte prediction heads across five downstream tasks. Acceptance rate measures the fraction of predicted bytes accepted. While MLP achieves higher raw acceptance, indicating that the model is highly confident about its prediction, it performs worse than LCA-MBP on the Pareto front.

We attribute this to the conditional independence assumption made by the MLP heads: each future token is predicted from a shared hidden state without conditioning on the other tokens being predicted in parallel. LCA, by contrast, uses transformer layers in the MBP head to condition each predicted byte on the previous segment’s context and on its position within the current segment, producing more coherent multi-byte predictions than independent MLP heads operating on the same shared hidden state. The result is a lower acceptance rate but higher end-to-end quality—a more useful operating point in practice.

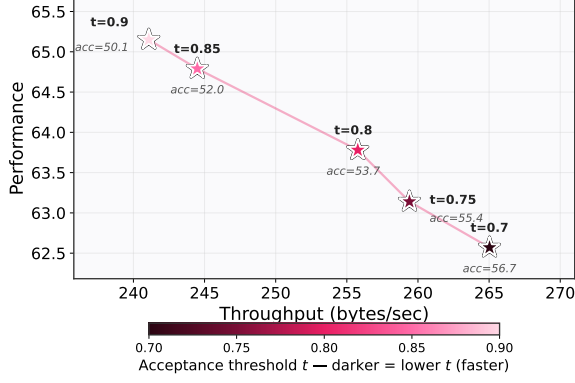


Figure 4: **Ablation on the acceptance threshold, averaged across DailySum, es-en, and fr-en.** Each star marks LCA evaluated at a different value of τ . Lowering τ admits more speculative tokens, increasing throughput and acceptance rate.

6 Discussion and Ablations

6.1 Ablation on the Acceptance Threshold τ

In Figure 4, we report average performance, throughput, and acceptance rate obtained by varying τ from 0.9 to 0.7, averaged across DailySum, es-en, and fr-en. As τ decreases, more speculative bytes are admitted: the acceptance rate rises from 50.1% to 56.7% and throughput improves by roughly 10%, while average performance declines by about 3 points. The drop is steep around $\tau = 0.75$, where the marginal throughput gain no longer justifies the loss in quality. Based on these results, we therefore can deduce that $\tau = 0.75$ offers a balanced trade-off between speed and downstream quality for further ablations.

6.2 Impact of Increasing Candidate Tokens Beyond Compression Rate

Our LCA-MBP model allows the number of speculative candidates n to be varied at inference time without retraining, by duplicating the last latent token as described in Line 10 of Algorithm 1. In this subsection, we study how this concept works by evaluating our model on multiple benchmarks using two candidate acceptance methods: (i) **Probability threshold** acceptance as discussed in Algorithm 1, and (ii) **speculative decoding** acceptance, where the candidate bytes from the multi-byte prediction head are verified with an extra forward pass through the next-byte prediction head. We employ the speculative decoding method as proposed in Leviathan et al. (2023).

Increment with Probability Threshold Acceptance: Figure 5 shows performance and through-

put at $\tau = 0.75$ as n varies from 3 to 7. We find that the optimal n can be task-specific: es-en and fr-en peak at $n = 7$ with about +3 points gain over $n = 3$, while DailySum peaks at $n = 6$ with +2.11 points. Beyond the peak, performance degrades as the model is asked to predict further past its trained horizon, and throughput drops monotonically due to falling acceptance rates (see Table 3). These results indicate that n can be tuned per task when using the threshold acceptance strategy.

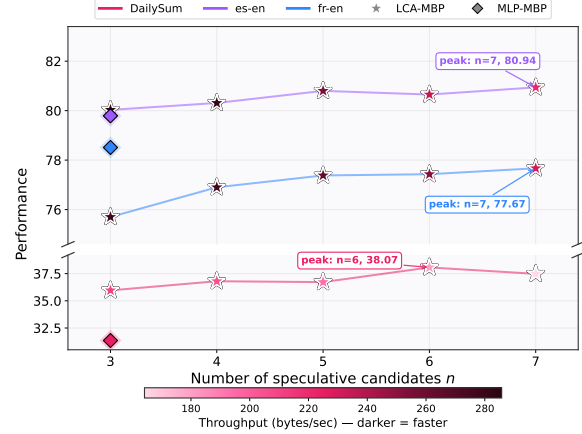


Figure 5: **Performance vs. Candidates (n at $\tau = 0.75$.)** Marker color encodes throughput (darker = faster; full values in Table 3). COMET scores (es-en, fr-en) scaled $\times 100$.

Increment with Speculative Decoding Acceptance: Unlike in probability threshold acceptance where increasing n changes both performance and inference speed, speculative decoding acceptance guarantees that all the model’s output matches those of decoding with the next-byte prediction head. In Figure 6, we find that downstream performance is constant across n , while throughput grows monotonically, gaining +29-37% between $n = 3$ and $n = 7-8$. LCA also overtakes the MLP-MBP baseline on all three tasks. Unlike MLP-MBP, whose number of prediction heads is fixed at training time, LCA allows n to be increased freely at inference. Under speculative decoding verification, n therefore acts as a free throughput knob: quality is preserved by construction while decoding speed scales with n .

Analysis of Accepted Bytes with Increasing Candidates (n): Given the ability to increase the number of candidate bytes in the MBP head beyond the trained compression rate, we investigate what fraction of candidate bytes are accepted from the MBP head alone over 100 decoding steps. Figure 7

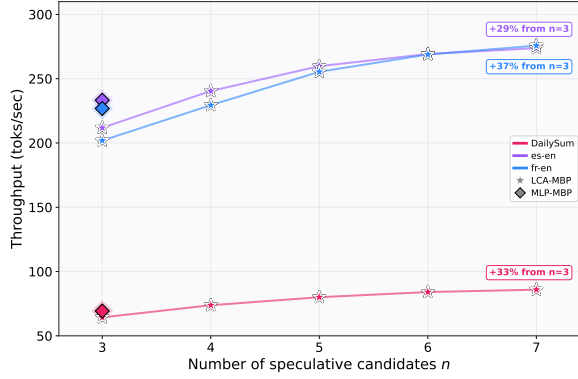


Figure 6: **Throughput vs. candidates n under speculative decoding acceptance.** Solid lines show LCA throughput growing monotonically with n on all three tasks while downstream performance is maintained (Table 4 in Appendix B).

shows that the number of accepted bytes varies widely from step to step: the model accepts anywhere from 0 to all 6 candidates, with a mean of 3.05 accepted per step and 15% of steps accepting the entire window. We also observe that no candidate byte is accepted at the first decoding step across multiple runs, as the model has not yet accumulated enough context to make confident multi-byte predictions.

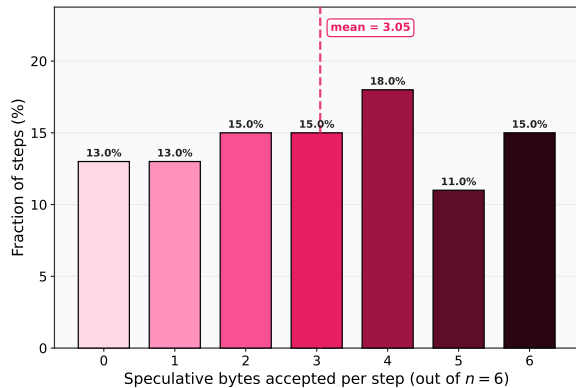


Figure 7: **Distribution of accepted speculative bytes over 100 decoding steps.** LCA accepts a mean of 3.05 of the 6 speculative candidates per step, and 15% of steps accept the full window, indicating that LCA’s multi-byte predictions are frequently coherent across an entire segment rather than accepted one byte at a time.

6.3 Improving Baseline Throughput with LCA Models

Here, we extend our analysis with speculative decoding acceptance to include generating candidate bytes that are then verified by an external baseline model. This ablation investigates how LCA can be used to accelerate a baseline model by increasing

its inference throughput while keeping the performance the same. In Figure 8, where we jointly use our LCA model as a drafter and an external FxT baseline as the verifier, we observe that verifying LCA’s multi-byte prediction head with the FxT model matches FxT’s downstream performance on all three tasks while achieving a $2.1\text{--}2.3\times$ speedup, even though both models are the same size.

This result highlights LCA’s role as a drop-in accelerator. Notably, the speedup does not come from a smaller draft model, as is typical in speculative decoding (Leviathan et al., 2023; Chen et al., 2023), but from predicting an entire latent token’s worth of bytes in parallel. This suggests that any hierarchical byte-level model can be paired with an LCA counterpart to gain substantial throughput improvements.

7 Related Work

7.1 Hierarchical Models and the Cost of Byte-Level Generation

Subword tokenization remains the dominant approach for language modeling, but fixed tokenizers introduce well-known limitations for rare words, domain shift, and evolving vocabularies (Sennrich et al., 2016). Byte-level models avoid these issues by operating directly on characters or UTF-8 bytes (Xue et al., 2022; Limisiewicz et al., 2024; Wang et al., 2024; Al-Rfou et al., 2018), but this comes at the cost of substantially longer sequences. As a result, byte-level models face a higher attention cost during training and slower autoregressive generation at inference time.

Recent hierarchical models address this problem by compressing byte or character sequences into shorter latent tokens before applying deeper sequence modeling. Some works explore sequence compression via pooling over fixed-size segments (Nawrot et al., 2022; Clark et al., 2022; Godey et al., 2022; Tay et al., 2022; Yu et al., 2023), while others employ boundary predictors that dynamically determine where to pool (Nawrot et al., 2023; Hwang et al., 2025). These boundary predictors are either trained in isolation, as in BLT (Pagnoni et al., 2025), or jointly optimized with the main language modeling objective (Hwang et al., 2025; Owodunni et al., 2025; Ahia et al., 2023). A related line of work studies how controlling the compression rate of a sequence affects performance across multiple languages and domains (Ahia et al., 2024; Owodunni et al., 2025; Limisiewicz et al., 2026).

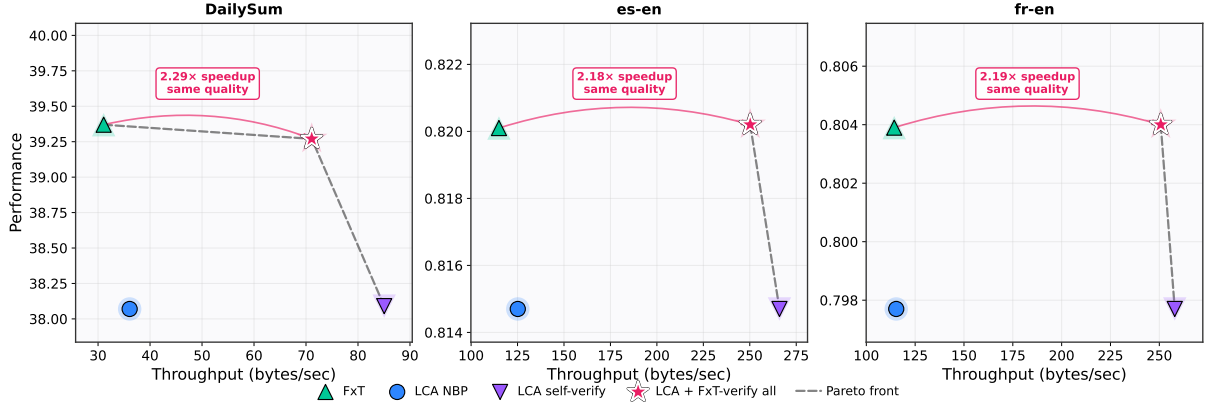


Figure 8: **Throughput vs. performance across four inference strategies.** We compare LCA’s next-byte head only, LCA’s self-verified multi-byte prediction, LCA with FxT verifying the MBP head, and the FxT baseline. External verification with FxT lets LCA match or exceed FxT’s performance while running 2.1-2.3 $\times$ faster across all three tasks with $n = 7$. See acceptance rate in Table 5 of Appendix B.

However, most hierarchical models use latent tokenization primarily to reduce sequence length or improve representation learning. At inference, they still follow standard autoregressive decoding, producing one byte at a time. Thus, while hierarchical models reduce the cost of processing byte sequences, they do not address the decoding bottleneck. Our work builds on hierarchical byte-level modeling, but uses learned latent tokens not only as compressed representations but also as the unit for parallel multi-byte prediction. A concurrent work, FastBLT (Kallini et al., 2026), uses diffusion language modeling to accelerate byte-level inference; unlike our approach, its boundary predictor is trained independently of the language model, as in BLT.

7.2 Improving Inference Speed with Multi-Token Prediction

Autoregressive language models generate one token per forward pass, making decoding latency a major bottleneck. Multi-token prediction (MTP) addresses this limitation by training models to predict multiple future tokens from the same context. Prior work has shown that predicting multiple future tokens can improve sample efficiency and accelerate generation (Gloeckle et al., 2024; Li et al., 2024; Gerontopoulos et al., 2026; Ankner et al., 2024). In practice, however, most MTP methods rely on additional prediction heads, auxiliary draft modules, or fixed future offsets, while others have not been explored for hierarchical models and multilingual settings (Grivas et al. (2025)).

Medusa (Cai et al., 2024), for example, attaches multiple decoding heads to a base model, and

similar embedded MTP approaches use separate lightweight heads for each future position (Cai et al., 2025). These designs share two limitations especially relevant to hierarchical byte-level models: the number of prediction heads grows with the number of future tokens, adding parameter cost, and fixed-offset prediction treats the next n tokens as a static horizon, even though byte sequences have variable local structure and may be better predicted as learned segments.

LCA addresses both limitations by tying multi-byte prediction to the segments produced by the hierarchical boundary predictor. A single decoder head equipped with the LCA mask predicts all bytes within a segment in parallel, conditioned only on previous segments. This removes the per-token parameter overhead of multi-head MTP and replaces fixed-offset prediction with variable-length, segment-aligned prediction.

8 Conclusion

We introduced LCA, a dynamic multi-byte prediction method for hierarchical byte-level LMs. Unlike prior MBP approaches that scale parameters with the number of tokens predicted in parallel, LCA uses a single decoder head and a boundary-aware attention mask to predict a variable number of bytes per step. Our method aligns MBP with the hierarchical structure already present in modern byte-level models. Across four downstream tasks, LCA lies on the Pareto front of performance and throughput on three of four. We show that latent tokens learned by hierarchical models can be reused as units of parallel generation, removing the need for separate per-token MBP heads.

Acknowledgments

This material is partially based upon work supported by OSU Kirwan Institute. We are also grateful to Amazon Research Awards (AWS), Ohio Supercomputing Center (OSC), UFIT Research Computing, and NAIRR Pilot Program (NAIRR250264) for providing the compute resources for this work.

Limitations

First, we conduct experiments at a single model scale (373M parameters) and training corpus, which lets us compare LCA against parameter-matched baselines under identical conditions; verifying that the gains persist at larger scales remains future work and would require more compute resources that are not available to us. Second, our evaluation covers four English-centric or English-paired tasks and does not include low-resource or morphologically rich languages, though LCA inherits its tokenization behavior directly from the boundary predictor and should transfer without architectural change. Finally, the speculative horizon n is task-dependent under threshold acceptance, which adds a small tuning cost; however, this cost disappears under speculative decoding verification (Figure 8), where n can be increased without changing model output.

References

- Orevaoghene Ahia, Sachin Kumar, Hila Gonen, Valentin Hofmann, Tomasz Limisiewicz, Yulia Tsvetkov, and Noah A. Smith. 2024. [MAGNET: Improving the multilingual fairness of language models with adaptive gradient-based tokenization](#). In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.
- Orevaoghene Ahia, Sachin Kumar, Hila Gonen, Jungo Kasai, David Mortensen, Noah Smith, and Yulia Tsvetkov. 2023. [Do all languages cost the same? tokenization in the era of commercial language models](#). In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 9904–9923, Singapore. Association for Computational Linguistics.
- Rami Al-Rfou, Dokook Choe, Noah Constant, Mandy Guo, and Llion Jones. 2018. [Character-level language modeling with deeper self-attention](#). In *AAAI Conference on Artificial Intelligence*.
- Zachary Ankner, Rishab Parthasarathy, Aniruddha Nrusimha, Christopher Rinard, Jonathan Ragan-Kelley, and William Brandon. 2024. [Hydra: Sequentially-dependent draft heads for medusa decoding](#). *arXiv preprint arXiv:2402.05109*.
- Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D Lee, Deming Chen, and Tri Dao. 2024. [Medusa: Simple llm inference acceleration framework with multiple decoding heads](#). *arXiv preprint arXiv:2401.10774*.
- Yuxuan Cai, Xiaozhuan Liang, Xinghua Wang, Jin Ma, Haijin Liang, Jinwen Luo, Xinyu Zuo, Lisheng Duan, Yuyang Yin, and Xi Chen. 2025. [Fastmtp: Accelerating llm inference with enhanced multi-token prediction](#). *arXiv preprint arXiv:2509.18362*.
- Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2023. [Accelerating large language model decoding with speculative sampling](#). *arXiv preprint arXiv:2302.01318*.
- Jonathan H. Clark, Dan Garrette, Iulia Turc, and John Wieting. 2022. [Canine: Pre-training an efficient tokenization-free encoder for language representation](#). *Transactions of the Association for Computational Linguistics*, 10:73–91.
- Anastasios Gerontopoulos, Spyridon Gidaris, and Nikos Komodakis. 2026. Multi-token prediction needs registers. *Advances in Neural Information Processing Systems*, 38:60189–60214.
- Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. 2024. [Better & faster large language models via multi-token prediction](#). *arXiv preprint arXiv:2404.19737*.
- Nathan Godey, Roman Castagné, Éric de la Clergerie, and Benoît Sagot. 2022. [MANTa: Efficient gradient-based tokenization for end-to-end robust language modeling](#). In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pages 2859–2870, Abu Dhabi, United Arab Emirates. Association for Computational Linguistics.
- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, and 1 others. 2024. [The llama 3 herd of models](#). *arXiv preprint arXiv:2407.21783*.
- Andreas Grivas, Lorenzo Loconte, Emile van Krieken, Piotr Nawrot, Yu Zhao, Euan Wielewski, Pasquale Minervini, Edoardo Ponti, and Antonio Vergari. 2025. [Fast and expressive multi-byte prediction with probabilistic circuits](#). *arXiv preprint arXiv:2511.11346*.
- Karl Moritz Hermann, Tomás Kociský, Edward Grefenstette, Lasse Espeholt, Will Kay, Mustafa Suleyman, and Phil Blunsom. 2015. [Teaching machines to read and comprehend](#). In *NIPS*, pages 1693–1701.
- Sukjun Hwang, Brandon Wang, and Albert Gu. 2025. [Dynamic chunking for end-to-end hierarchical sequence modeling](#). *arXiv preprint arXiv:2507.07955*.
- Julie Kallini, Artidoro Pagnoni, Tomasz Limisiewicz, Gargi Ghosh, Luke Zettlemoyer, Christopher Potts, Xiaochuang Han, and Srinivasan Iyer. 2026.

- Fast byte latent transformer. *arXiv preprint arXiv:2605.08044*.
- John Kirchenbauer, Abhimanyu Hans, Brian Bartoldson, Micah Goldblum, Ashwinee Panda, and Tom Goldstein. 2026. Multi-token prediction via self-distillation. *arXiv preprint arXiv:2602.06019*.
- Taku Kudo. 2018. Subword regularization: Improving neural network translation models with multiple subword candidates. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 66–75.
- Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V. Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, Yuling Gu, Saumya Malik, Victoria Graf, Jena D. Hwang, Jiangjiang Yang, Ronan Le Bras, Oyvind Tafjord, Chris Wilhelm, Luca Soldaini, and 4 others. 2024. Tülu 3: Pushing frontiers in open language model post-training.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In *International conference on machine learning*, pages 19274–19286. PMLR.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2024. Eagle: Speculative sampling requires rethinking feature uncertainty. *arXiv preprint arXiv:2401.15077*.
- Tomasz Limisiewicz, Terra Blevins, Hila Gonen, Orevaoghene Ahia, and Luke Zettlemoyer. 2024. **MYTE: Morphology-driven byte encoding for better and fairer multilingual language modeling**. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15059–15076, Bangkok, Thailand. Association for Computational Linguistics.
- Tomasz Limisiewicz, Artidoro Pagnoni, Srinu Iyer, Mike Lewis, Sachin Mehta, Alisa Liu, Margaret Li, Gargi Ghosh, and Luke Zettlemoyer. 2026. Compute optimal tokenization. *arXiv preprint arXiv:2605.01188*.
- Anton Lozhkov, Loubna Ben Allal, Leandro von Werra, and Thomas Wolf. 2024. **Fineweb-edu: the finest collection of educational content**.
- Mikhail Menschikov, Alexander Kharitonov, Maiia Kotyga, Vadim Porvatov, Anna Zhukovskaya, David Kagramanyan, Egor Shvetsov, and Evgeny Burnaev. 2025. Beyond early-token bias: Model-specific and language-specific position effects in multilingual llms. *arXiv preprint arXiv:2505.16134*.
- Sabrina J Mielke, Zaid Alyafeai, Elizabeth Salesky, Colin Raffel, Manan Dey, Matthias Gallé, Arun Raja, Chenglei Si, Wilson Y Lee, Benoît Sagot, and 1 others. 2021. Between words and characters: A brief history of open-vocabulary modeling and tokenization in nlp. *arXiv preprint arXiv:2112.10508*.
- Piotr Nawrot, Jan Chorowski, Adrian Lancucki, and Edoardo Maria Ponti. 2023. Efficient transformers with dynamic token pooling. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6403–6417.
- Piotr Nawrot, Szymon Tworowski, Michał Tyrolski, Lukasz Kaiser, Yuhuai Wu, Christian Szegedy, and Henryk Michalewski. 2022. **Hierarchical transformers are more efficient language models**. In *Findings of the Association for Computational Linguistics: NAACL 2022*, pages 1559–1571, Seattle, United States. Association for Computational Linguistics.
- Pit Neiteimeier, Björn Deiseroth, Constantin Eichenberg, and Lukas Balles. 2025. Hierarchical autoregressive transformers: Combining byte- and word-level processing for robust, adaptable language models. *arXiv preprint arXiv:2501.10322*.
- Abraham Toluase Owodunni, Orevaoghene Ahia, and Sachin Kumar. 2025. Flexitokens: Flexible tokenization for evolving language models. *arXiv preprint arXiv:2507.12720*.
- Artidoro Pagnoni, Ramakanth Pasunuru, Pedro Rodriguez, John Nguyen, Benjamin Muller, Margaret Li, Chunting Zhou, Lili Yu, Jason E Weston, Luke Zettlemoyer, and 1 others. 2025. Byte latent transformer: Patches scale better than tokens. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9238–9258.
- Weizhen Qi, Yu Yan, Yeyun Gong, Dayiheng Liu, Nan Duan, Jiusheng Chen, Ruofei Zhang, and Ming Zhou. 2020. **ProphetNet: Predicting future n-gram for sequence-to-sequence pre-training**. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 2401–2410, Online. Association for Computational Linguistics.
- Siva Reddy, Danqi Chen, and Christopher D. Manning. 2019. **CoQA: A conversational question answering challenge**. *Transactions of the Association for Computational Linguistics*, 7:249–266.
- Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. **Neural machine translation of rare words with subword units**. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725, Berlin, Germany. Association for Computational Linguistics.
- Kevin Slagle. 2024. Spacebyte: Towards deleting tokenization from large language modeling. *Advances in Neural Information Processing Systems*, 37:124925–124950.
- Mitchell Stern, Noam Shazeer, and Jakob Uszkoreit. 2018. Blockwise parallel decoding for deep autoregressive models. *Advances in Neural Information Processing Systems*, 31.

Yi Tay, Vinh Q. Tran, Sebastian Ruder, Jai Gupta, Hyung Won Chung, Dara Bahri, Zhen Qin, Simon Baumgartner, Cong Yu, and Donald Metzler. 2022. [Charformer: Fast character transformers via gradient-based subword tokenization](#). In *International Conference on Learning Representations*.

Junxiong Wang, Tushaar Gangavarapu, Jing Nathan Yan, and Alexander M Rush. 2024. Mambabyte: Token-free selective state space model. *arXiv preprint arXiv:2401.13660*.

Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. 2022. [ByT5: Towards a token-free future with pre-trained byte-to-byte models](#). *Transactions of the Association for Computational Linguistics*, 10:291–306.

Lili Yu, Dániel Simig, Colin Flaherty, Armen Aghajanyan, Luke Zettlemoyer, and Mike Lewis. 2023. Megabyte: Predicting million-byte sequences with multiscale transformers. *Advances in Neural Information Processing Systems*, 36:78808–78823.

Biao Zhang, Philip Williams, Ivan Titov, and Rico Senrich. 2020. [Improving massively multilingual neural machine translation and zero-shot translation](#). In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 1628–1639, Online. Association for Computational Linguistics.

Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. 2023. [Instruction-following evaluation for large language models](#). *Preprint*, arXiv:2311.07911.

Appendix

A General Architecture, Training Configurations, and Hyperparameters

Reading Figure 1. We use a consistent visual encoding throughout the figure. The uniform red boxes denote the processed bytes sequence. In the boundary predictor, multi-colored bands indicate tokenization into segments, with bytes sharing a color belonging to the same segment. Deeper shades (green, orange, blue, yellow) mark latent tokens, which are pooled aggregates of the bytes in each segment, and rotated color blocks indicate positions where rotary positional embeddings (RoPE) have been applied. Broken outlines on the latent tokens after the LM module signal that those tokens have been contextualized by the LM transformer stack. Finally, module backgrounds encode the attention mask used by each module: yellow for standard causal attention and orange for the LCA mask.

A.1 Hyperparameters

Table 2 summarizes the architectural configurations used for the main byte-level model variants. All models use the same optimizer family and training hyperparameters unless explicitly noted elsewhere. For the MLP-MBP configuration trained at compression rate 5, the MB decoder depth is increased from 3 to 5 layers.

Model	Encoder	LM Module Module	Decoder NB NT	Decoder MB MB
LlamaByte	22	0	0	0
SpaceByte	2	16	4	0
FxT	2	16	4	0
MLP-MBP	2	16	2	3
LCA-MBP	2	16	2	2

Table 2: Architectural configurations used across model variants. Unless otherwise stated, all models use the same hidden dimension, training hyperparameters, and optimizer settings. For MLP-MBP trained with compression rate 5, the MB decoder uses 5 layers instead of 3.

B Extended Results

French Translation Results. Figure 9 reports performance versus throughput on French→English translation. LCA lies on the Pareto front, achieving substantially higher BLEU than MLP-MBP at similar throughput. The trend mirrors the Spanish→English result in Figure 3.

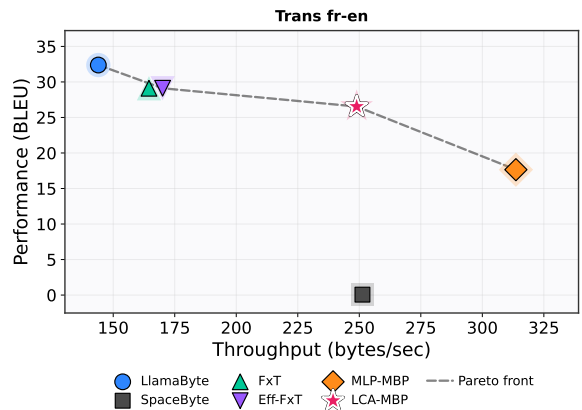


Figure 9: **Performance vs. throughput on French-English translation.** LCA (★) lies on the Pareto front.

Throughput for Candidate Bytes n Ablation; Table 3 reports throughput (tokens/second) for the n -ablation in Figure 5. Throughput decreases monotonically with n on all three tasks, as larger

speculative windows incur more rejected candidates per step. The drop is steepest on DailySum (-22% from $n = 3$ to $n = 7$), reflecting its lower baseline throughput and tighter sensitivity to rejection overhead.

n	DailySum	es-en	fr-en
3	210.42	285.75	278.21
4	205.42	283.47	278.13
5	194.50	254.57	263.99
6	181.24	234.55	252.31
7	164.18	221.09	230.99

Table 3: Throughput (bytes/sec) of LCA-MBP across speculative-candidate counts $n \in \{3, 4, 5, 6, 7\}$ at acceptance threshold $\tau = 0.75$. Throughput decreases monotonically with n on all three tasks, as larger speculative windows incur more rejected candidates per step.

Performance and Acceptance Rate with Speculative Decoding: Here we report acceptance rate and performance at $n = 3$ for our speculative decoding results in Table 4. We dropped KV caching for all our results that use this method. We note that LCA-MBP outperforms MLP-MBP on multiple tasks despite achieving a lower acceptance rate.

Model	IfEval	CoQA	DailySum	es-en	fr-en	Avg
<i>Performance</i>						
MLP-MBP	46.54	20.85	32.73	0.8178	0.8064	–
LCA-MBP	44.97	22.11	38.09	0.8147	0.7977	–
<i>Acceptance rate (%)</i>						
MLP-MBP	71.80	90.15	90.79	91.82	91.82	87.28
LCA-MBP	51.17	73.93	81.22	79.85	79.85	73.20

Table 4: Performance and acceptance rate under speculative decoding acceptance. Performance is constant across the number of speculative candidates n ; acceptance rates are reported at $n = 3$.

Throughput and Acceptance rate for LCA with External Verification: Table 5 reports the token acceptance rates for the external verification experiment in §6.3. FxT and LCA NBP accept every token by construction, since they perform standard next-byte decoding without speculation. Under speculative decoding, LCA Self-verify and LCA FxT-verify accept 78-84% of speculative bytes across the three tasks, meaning that most multi-byte predictions are confirmed on the first pass and only a minority require a fallback to the verifier.

Model	DailySum	es-en	fr-en
FxT	100.00	100.00	100.00
LCA NBP	100.00	100.00	100.00
LCA Self-verify	46.36	45.85	45.85
LCA FxT-verify	47.97	50.77	52.04

Table 5: Token acceptance rates (%) across three tasks under self and external (FxT) verification using $n = 7$ candidate bytes.